
System Call

Study_Material

CS3101: Operating System Lab 3



Contents

1	System Calls	1
2	How System Calls Work	1
3	Examples of System Calls in Linux	1
4	Task 1 Solution	1
4.1	xv6	1
4.2	Use the following commands to install xv6:	2
4.3	Build and start xv6 using the following commands:	2
4.4	Install the required compilation, emulation, and debugging tools for building and running xv6 on the RISC-V architecture by executing the following command in the Ubuntu terminal:	2
4.5	After completing the installation, verify that the RISC-V compiler, binary utilities, QEMU emulator, and GNU debugger are installed correctly by executing the following commands in the Ubuntu terminal:	2
4.6	Create a Git working branch:	3
4.7	Confirm the branch and initial status:	3
4.8	Assign an unused system-call number	3
4.9	Implement the Kernel-Side Handler	4
4.10	Register the Handler in the Kernel Dispatcher	5
4.11	Add the User-Space Function Prototype	5
4.12	Generate the User-Space System-Call Stub	5
4.13	Create the User-Space Test Program	6
4.14	Add <code>addtest</code> to the xv6 File-System Image	6
4.15	Verify the Complete Registration Chain Before Building	6
4.16	Compile the Modified xv6 Source	7
4.17	Boot xv6 and Test the New System Call	7
4.18	Design the Shared System-Information Header	7
4.19	Assign a new System-Call Number	8
4.20	Count the Available Physical Memory	8
4.20.1	Pseudocode	8
4.21	Count Active Processes	9
4.21.1	Design the Active-Process Counting Helper	9
4.22	Declare the Kernel Helper Functions	10
4.23	Implement the Kernel-Side <code>sysinfo</code> Handler	10
4.24	Register <code>sysinfo</code> in the Kernel Dispatcher	11

4.25 Add the User-Space Prototype	11
4.26 Generate the User-Space System-Call Stub	11
4.27 Create the <code>sysinfo</code> Test Program	11
4.28 Add <code>sysinfotest</code> to the File-System Image	11
4.29 Verify the Complete <code>sysinfo</code> Registration Chain	11
4.30 Compile the Modified xv6 Source	11
4.31 Boot xv6 and Test the New System Call	11

1 System Calls

System calls in Linux provide an interface between a running process and the operating system's kernel. They allow user-level processes to request kernel services, such as accessing hardware, managing memory, controlling processes, and performing file operations. Essentially, system calls enable user applications to communicate with the operating system.

1. System calls allow programs to interact with the operating system's kernel.
2. Linux provides hundreds of system calls for purposes such as file management, process control, memory management, and inter-process communication.
3. Common Linux system calls include `open()`, `read()`, `write()`, `fork()`, `exec()`, and `exit()`.

2 How System Calls Work

When a user program needs to perform a task that requires kernel-level access, such as reading a file or creating a process, it issues a system call. The CPU switches from user mode to kernel mode, and the corresponding system-call handler in the kernel is executed. After the requested operation has been completed, the kernel returns control to the program, and the CPU switches back to user mode.

3 Examples of System Calls in Linux

1. **Open a File `open()`:** To open a file, a process must request access from the kernel using the `open()` system call.
2. **Read from a File `read()`:** The `read()` system call is used to read data from a file.
3. **Creating a New Process `fork()`:** The `fork()` system call is used to create a new process by duplicating the current process.

4 Task 1 Solution

4.1 xv6

xv6 is a modern reimplementaion of Sixth Edition Unix in ANSI C for multiprocessor x86 and RISC-V systems. It was created for educational purposes in MIT's Operating System Engineering course in 2006. MIT's Operating System Engineering course formerly used the original V6 source code. xv6 was created as a modern replacement because PDP-11 machines are not widely available, and the original operating system was written in archaic pre-ANSI C. Unlike Linux or BSD, xv6 is simple enough to cover in a semester, yet still contains the important concepts and organization of Unix. You can explore more about xv6 using the following link:

<https://pdos.csail.mit.edu/6.828/2012/xv6.html>

4.2 Use the following commands to install xv6:

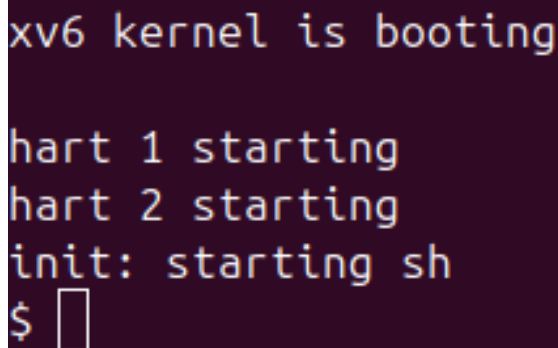
```
cd "$HOME"

git clone https://github.com/mit-pdos/xv6-riscv.git

cd xv6-riscv
```

4.3 Build and start xv6 using the following commands:

```
make clean
make
make qemu
```

A screenshot of a terminal window with a dark purple background. The text is displayed in a monospaced font with a rainbow color gradient. The output shows the xv6 kernel booting, followed by two harts starting, the init process starting a shell, and a prompt character '\$' followed by a cursor box.

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ █
```

Figure 1: Output 1

4.4 Install the required compilation, emulation, and debugging tools for building and running xv6 on the RISC-V architecture by executing the following command in the Ubuntu terminal:

```
sudo apt install -y \
    build-essential \
    gdb-multiarch \
    qemu-system-misc \
    gcc-riscv64-linux-gnu \
    binutils-riscv64-linux-gnu
```

4.5 After completing the installation, verify that the RISC-V compiler, binary utilities, QEMU emulator, and GNU debugger are installed correctly by executing the following commands in the Ubuntu terminal:

```
riscv64-linux-gnu-gcc --version

riscv64-linux-gnu-objdump --version

qemu-system-riscv64 --version

gdb-multiarch --version | head -n 1
```

```
shubhamkant66@shubhamkant66:~/xv6-riscv$ riscv64-linux-gnu-gcc --version
riscv64-linux-gnu-gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

shubhamkant66@shubhamkant66:~/xv6-riscv$ riscv64-linux-gnu-objdump --version
GNU objdump (GNU Binutils for Ubuntu) 2.42
Copyright (C) 2024 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or (at your option) any later version.
This program has absolutely no warranty.

shubhamkant66@shubhamkant66:~/xv6-riscv$ qemu-system-riscv64 --version
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.18)
Copyright (c) 2003-2023 Fabrice Bellard and the QEMU Project developers
shubhamkant66@shubhamkant66:~/xv6-riscv$ gdb-multiarch --version | head -n 1
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
shubhamkant66@shubhamkant66:~/xv6-riscv$
```

Figure 2: Output 2

4.6 Create a Git working branch:

```
git switch -c lab-simple-system-call

If this branch already exists, use:
git switch lab-simple-system-call
```

4.7 Confirm the branch and initial status:

```
git branch --show-current

git status --short
```

```
shubhamkant66@shubhamkant66:~/xv6-riscv$ git switch -c lab-simple-system-call
fatal: a branch named 'lab-simple-system-call' already exists
shubhamkant66@shubhamkant66:~/xv6-riscv$ git switch lab-simple-system-call
Already on 'lab-simple-system-call'
shubhamkant66@shubhamkant66:~/xv6-riscv$ git branch --show-current
lab-simple-system-call
shubhamkant66@shubhamkant66:~/xv6-riscv$ git status --short
shubhamkant66@shubhamkant66:~/xv6-riscv$
```

Figure 3: Output 3

4.8 Assign an unused system-call number

The current official xv6-RISC-V tree uses number 22 for `SYS_sync`. Therefore, the next unused number is 23. If the local course version differs, select the next unused positive number and use it consistently in every relevant file.

Open the system-call number header file using the following command:

```
nano kernel/syscall.h
```

Add the following definition after the existing system-call number definitions:

```
#define SYS_<function_name> unused_number
```

Verify that the new system-call definition has been added correctly and confirm that system-call number has not been assigned more than once:

```
grep -n "SYS_<function_name>" kernel/syscall.h

grep -nE '^#define[[:space:]]+SYS_[A-Za-z0-9_]+[[:space:]]+23$' \
    kernel/syscall.h
```

4.9 Implement the Kernel-Side Handler

Open the process-related system-call implementation file using the following command:

```
nano kernel/sysproc.c
```

The `kernel/sysproc.c` file is a C source file inside the `kernel` directory of `xv6` that contains the kernel-side functions for handling several system calls, particularly those related to processes, such as `fork()`, `exit()`, `wait()`, `sleep()`, and `getpid()`. A user program cannot directly execute protected kernel operations, so it requests the kernel to perform them through a system call.

Add the following function at the end of the file:

Use the following pseudocode as a guide to implement the kernel-side system-call handler:

```
BEGIN system-call handler

    Create two integer variables for the input values.

    Retrieve argument 0 using xv6's integer-argument
    extraction helper.

    Retrieve argument 1 using the same helper.

    IF either argument is outside the permitted range THEN
        Return the specified failure value.
    ELSE
        Compute and return the sum of the two arguments.
    END IF

END system-call handler
```

- Use `argint()` to retrieve integer arguments.

Students must translate this pseudocode into valid C code by following the structure of existing system-call handlers in `kernel/sysproc.c`.

Verify the kernel-side system-call handler using the following command:

```
grep -n -A18 "sys_<function_name>" kernel/sysproc.c
```

4.10 Register the Handler in the Kernel Dispatcher

Open the system-call dispatcher using the following command:

```
nano kernel/syscall.c
```

The `syscall.c` file is a C source file in an operating system kernel that helps manage system calls. A system call is a request made by a user program when it needs the kernel to perform a protected operation, such as reading a file, displaying output, creating a process, or accessing hardware. Because normal programs cannot directly access kernel resources, they send a system-call request containing a unique system-call number. The code associated with `syscall.c` identifies this number, finds the correct kernel function, passes the required arguments to it, and returns the result to the user program.

First, add the handler declaration beside the other `extern` declarations:

```
extern uint64 sys_<function_name>;
```

Second, add the handler to the `syscalls[]` dispatch table:

```
[SYS_<function_name>] sys_<function_name> ,
```

Verify that both lines are present using the following command:

```
grep -n "sys_<function_name>" kernel/syscall.c
```

4.11 Add the User-Space Function Prototype

Open the user-space header file from the root directory of the xv6 repository using the following command:

```
nano user/user.h
```

Add the following function prototype beside the other system-call declarations:

```
<return_type> <function_name>(parameters);
```

Verify that the function prototype has been added correctly:

```
grep -n "<function_name>" user/user.h
```

4.12 Generate the User-Space System-Call Stub

The `user/usys.pl` file is a Perl script used specifically in the xv6 operating system, rather than a standard file in the general Linux kernel. It automatically generates small assembly-language functions called system-call stubs for user programs. When a user program calls a function such as `fork()`, `write()`, or the custom `addnumbers()`, the generated stub places the corresponding system-call number in the appropriate processor register and executes the RISC-V `ecall` instruction to transfer control from user mode to the kernel.

Open the `user/usys.pl` script using the following command:

```
nano user/usys.pl
```

Add the following line beside the other `entry(...)` statements:

```
entry("function_name");
```

Verify that the new entry has been added correctly:

```
grep -n "<function_name>" user/usys.pl
```

4.13 Create the User-Space Test Program

Remain in the root directory of the xv6 repository and create the test program at the correct path using the following command:

```
nano user/<test_function_name>.c
```

Insert the following C program into the file:

```
/*
 * test_function_name.c
 *
 * User-space test program for the addnumbers system call.
 */

#include "kernel/types.h"
#include "user/user.h"

/*
 * Execute one test and compare the returned result with
 * the expected result.
 *
 * Return 0 when the test passes.
 * Return 1 when the test fails.
 */
```

4.14 Add addtest to the xv6 File-System Image

Open the top-level **Makefile**, located in the root directory of the **xv6-riscv** repository, using the following command:

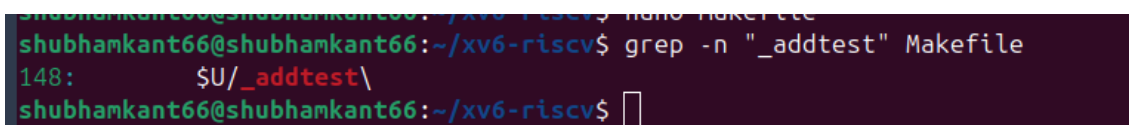
```
nano ~/xv6-riscv/Makefile
```

Find the **UPROGS** list and add the following line using the same indentation and formatting as the surrounding entries:

```
$U/_<test_function_name>\
```

Save the **Makefile** and verify the modification using the following command:

```
grep -n "_<test_function_name>" Makefile
```



```
shubhamkant66@shubhamkant66:~/xv6-riscv$ nano Makefile
shubhamkant66@shubhamkant66:~/xv6-riscv$ grep -n "_addtest" Makefile
148: $U/_addtest\
shubhamkant66@shubhamkant66:~/xv6-riscv$
```

Figure 4: Output 9

4.15 Verify the Complete Registration Chain Before Building

Run the following commands from the root directory of the **xv6-riscv** repository:

```
grep -n "SYS_<function_name>" kernel/syscall.h
grep -n "sys_<function_name>" kernel/sysproc.c
grep -n "sys_<function_name>" kernel/syscall.c
```

```
grep -n "<function_name>" user/user.h
grep -n "<function_name>" user/usys.pl
grep -n "<test_function_name>" Makefile
test -f user/<test_function_name>.c && echo "user/<test_function_name>.c exists "
```

Check the list of changed files and verify that the files do not contain whitespace errors:

```
git status --short
git diff --check
```

If the `git diff -check` command produces no output, no whitespace errors were detected.

4.16 Compile the Modified xv6 Source

Run the following build commands in the Ubuntu host terminal from the root directory of the `xv6-riscv` repository:

```
make clean
make
```

4.17 Boot xv6 and Test the New System Call

Start xv6 by running the following command in the Ubuntu host terminal from the root directory of the `xv6-riscv` repository:

```
cd ~/xv6-riscv
make qemu
```

Wait until xv6 finishes booting and displays its shell prompt. At the xv6 shell prompt, run the custom test program:

```
<test_function_name>
```

4.18 Design the Shared System-Information Header

The `sysinfo` system call must exchange information between the xv6 kernel and a user program. Therefore, both kernel-space and user-space source files require a common definition of the structure used to carry this information.

Create and open the shared header file using:

```
nano kernel/sysinfo.h
```

What must the header contain?

Design a structure that can hold the following two system statistics:

1. The total amount of currently available physical memory, expressed in bytes.
2. The number of process-table entries whose state is not `UNUSED`.

Header-File Design Hints

Your header should contain:

- an include guard to prevent multiple definitions;
- one structure representing the system information;
- a field named `freemem` for the free-memory count;
- a field named `nproc` for the process count; and
- the closing directive for the include guard.

```
#ifndef <CHOOSE_A_UNIQUE_HEADER_GUARD>
#define <REPEAT_THE_SAME_HEADER_GUARD>

struct <SYSTEM_INFORMATION_STRUCTURE_NAME> {
    <XV6_64_BIT_UNSIGNED_TYPE> freemem;
    <XV6_64_BIT_UNSIGNED_TYPE> nproc;
};

#endif
```

4.19 Assign a new System-Call Number

4.20 Count the Available Physical Memory

The `sysinfo` system call must report the amount of physical memory that is currently available for allocation. In xv6, `kernel/kalloc.c` implements a simple physical-page allocator through functions such as `kalloc()` and `kfree()`.

In contrast, the Linux kernel uses a more sophisticated memory-management system distributed across multiple source files. These include `mm/page_alloc.c` for physical-page allocation, `mm/slub.c` for kernel-object allocation, and `mm/vmalloc.c` for virtually contiguous memory allocation.

Open the physical-page allocator using:

```
nano kernel/kalloc.c
```

4.20.1 Pseudocode

Add the function to count available memory.

```
FUNCTION count_available_memory
    total_bytes ← 0

    acquire the physical-memory allocator lock

    current_page ← first node of the free list

    WHILE current_page is not null
        increase total_bytes by the size of one page
        move current_page to the next free-list node
    END WHILE
```

```
    release the physical-memory allocator lock
```

```
    RETURN total_bytes
```

```
END FUNCTION
```

4.21 Count Active Processes

Open the process implementation file using the following command:

```
nano kernel/proc.c
```

In xv6-RISC-V, `proc.c` is the kernel source file responsible for managing processes. A process is a program that is currently being executed, and `proc.c` file contains the main functions required to create, schedule, run, sleep, wake up, and terminate processes. It maintains the process table, where each process is represented by a `struct proc` containing information such as its process ID, current state, memory page table, open files, parent process, and kernel stack. Functions such as `allocproc()` create a process entry, `fork()` creates a child process, `exit()` terminates a process, `wait()` waits for a child process, `scheduler()` selects a runnable process, and `sleep()` and `wakeup()` control process waiting.

Add the following function at file scope, for example after the existing `myproc()` function:

4.21.1 Design the Active-Process Counting Helper

The `sysinfo` system call requires the number of process-table entries that are currently in use. An entry is counted when its state is different from `UNUSED`.

The required condition is:

$$\text{An entry contributes to the count} \iff \text{entry.state} \neq \text{UNUSED}.$$

```
FUNCTION count_process_entries_in_use
```

```
    count <- zero
```

```
    current <- first process-table entry
```

```
    WHILE current is before the end of the process table
```

```
        acquire the lock belonging to current
```

```
        IF the state of current is not UNUSED
```

```
            increase count by one
```

```
        END IF
```

```
        release the lock belonging to current
```

```
        advance current to the next entry
```

```
    END WHILE
```

```
    RETURN count
```

```
END FUNCTION
```

Locking requirement: The state of a process may change while the kernel is running. Therefore, the state must be examined while holding the spinlock belonging to that process-table entry.

Use the following lock discipline:

acquire lock → read state → update local count → release lock.

Only one process lock should be held at a time. Release the current lock before examining the next process-table entry. Failing to release a lock can cause a kernel deadlock.

4.22 Declare the Kernel Helper Functions

The free-memory and process-counting helpers are implemented in separate kernel source files. However, the system-call handler will need to call both functions from another source file. Their function prototypes must therefore be added to the kernel's shared declaration header.

Open the kernel function-declaration header using:

```
nano kernel/defs.h
```

Locate the `// kalloc.c` group and add the declaration of the **free-memory** helper function:

```
<return type> <free-memory helper name>(<parameter list >);
```

Locate the `// proc.c` group and add the declaration of the **process-count** helper helper function:

```
<return type> <process-count helper name>(<parameter list >);
```

4.23 Implement the Kernel-Side `sysinfo` Handler

Open the file containing the process-related system-call implementations using the following command:

```
nano kernel/sysproc.c
```

Add the shared `sysinfo.h` header alongside the existing header-file inclusions near the beginning of `kernel/sysproc.c`:

```
#include "sysinfo.h"
```

Use the following pseudocode as an implementation guide:

```
BEGIN system-information handler
```

```
    Declare a variable for the user-space destination address
```

```
    Declare a local system-information structure
```

```
    Retrieve system-call argument 0 as a user-space address
```

```
    Store the available-memory value in the structure
```

```
    Store the active-process count in the structure
```

```
    Safely copy the structure from kernel space to user space
        using the current process's page table
```

```
    IF the copy operation fails
```

```
        Return an error
    END IF
```

```
    Return success
```

```
END system-information handler
```

4.24 Register `sysinfo` in the Kernel Dispatcher

4.25 Add the User-Space Prototype

4.26 Generate the User-Space System-Call Stub

4.27 Create the `sysinfo` Test Program

4.28 Add `sysinfotest` to the File-System Image

4.29 Verify the Complete `sysinfo` Registration Chain

4.30 Compile the Modified `xv6` Source

4.31 Boot `xv6` and Test the New System Call